



SIPEN

Sistema Integrado de Gestão — IPPenha

 Documentação de Arquitetura e Referência Técnica

VERSÃO
v6.30.36

DATA
Mai 2026

MÓDULOS
22 módulos

Índice

1 Visão Geral do Sistema

1.1 Propósito e contexto

1.2 Stack tecnológica

1.3 Diagrama de arquitetura

2 Estrutura do Projeto

2.1 Organização de pastas

2.2 Camada Core

2.3 Padrão de módulo

3 Módulos do Sistema

4 Perfis e Permissões

5 Banco de Dados (Supabase)

5.1 Tabelas principais

5.2 Row Level Security (RLS)

6 Edge Functions (Serverless)

7 Integração WhatsApp

7.1 Arquitetura geral

7.2 Fluxo de envio (frontend → BotConversa)

7.3 Fluxo de recebimento (webhook + IA)

7.4 Módulos com notificações WA

7.5 Tabelas WhatsApp

8 Autenticação e Segurança

9 Deploy e Infraestrutura

10 Referências e Credenciais



1. Visão Geral do Sistema

1.1 Propósito e Contexto

O **SIPEN** (Sistema Integrado de Gestão — IPPenha) é uma aplicação web progressiva (PWA) desenvolvida para centralizar a gestão administrativa, pastoral, financeira e ministerial da **Igreja Presbiteriana de Penha**.

O sistema foi construído como uma **SPA (Single Page Application)** em Vanilla JavaScript, sem frameworks como React ou Vue, utilizando o Supabase como backend completo (banco de dados PostgreSQL, autenticação, storage e Edge Functions).

22 Módulos

Gestão completa: membros, finanças, agenda, demandas, conselho, pastoral, ministerial, infraestrutura, jurídico e mais.

Zero Dependências Frontend

Vanilla JS puro. Nenhum framework SPA. HTML, CSS e JS nativos com módulos IIFE por escopo.

WhatsApp Integrado

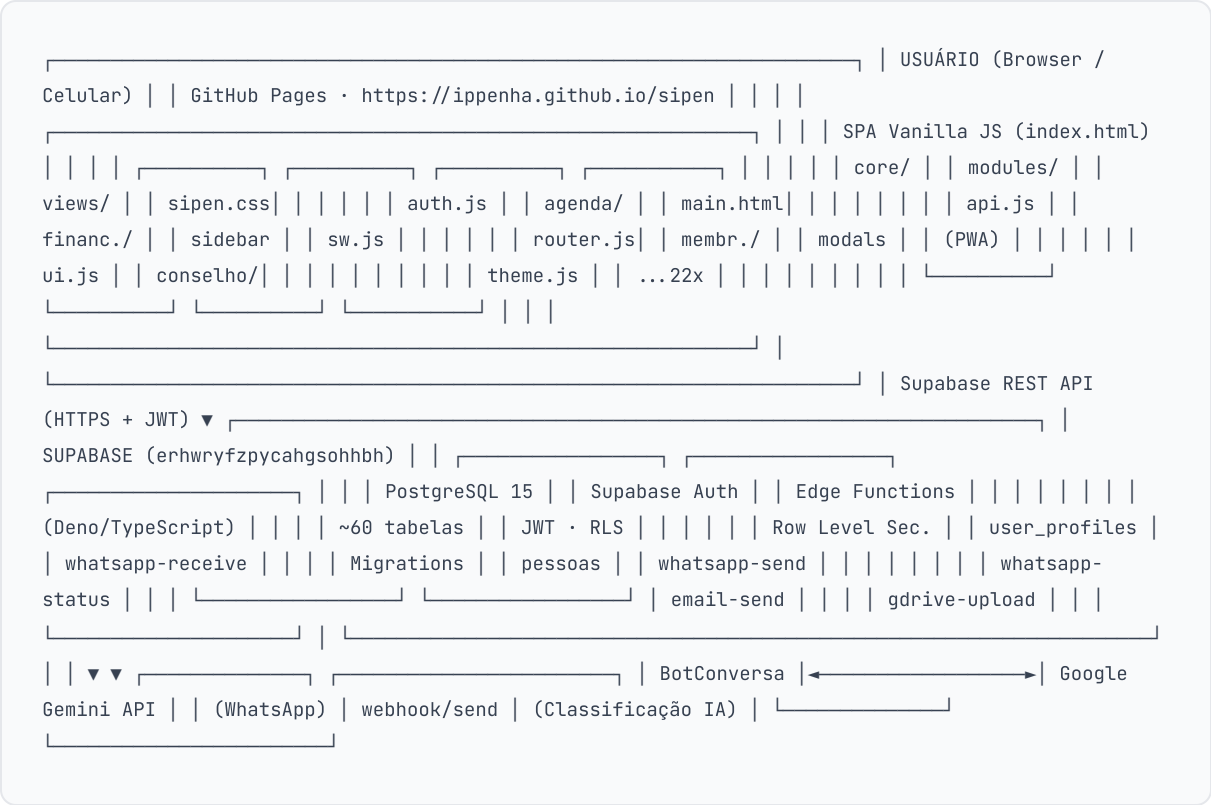
Envio e recebimento via BotConversa + IA Gemini para classificação automática de demandas.

1.2 Stack Tecnológica

Camada	Tecnologia	Versão / Detalhe	Responsabilidade
Frontend	HTML5 + CSS3 + JavaScript ES2022	Vanilla (sem framework)	SPA com roteamento client-side, módulos IIFE
PWA	Service Worker + Web App Manifest	Cache offline	Instalação no celular, uso offline parcial
Backend / DB	Supabase (PostgreSQL 15)	Projeto: erhwryfzpycahgsohhbh	Banco, autenticação JWT, RLS, Storage
Serverless	Supabase Edge Functions (Deno)	TypeScript	WhatsApp, e-mail, Google Drive, status WA
Autenticação	Supabase Auth	JWT + session localStorage	Login, perfis, RLS automático
WhatsApp	BotConversa API	REST + Webhook	Envio/recebimento de mensagens WA
IA	Google Gemini Flash Lite	gemini-2.5-flash-lite	Classificação de mensagens WA em demandas
E-mail	Nodemailer (SMTP)	Edge Function email-send	Notificações por e-mail

Camada	Tecnologia	Versão / Detalhe	Responsabilidade
Hosting	GitHub Pages	Branch main	Hospedagem estática do frontend
Versionamento	Git / GitHub	Repositório privado	Controle de versão e deploy automático

1.3 Diagrama de Arquitetura





2. Estrutura do Projeto

2.1 Organização de Pastas

sipen/ |— index.html ← ponto de entrada único da SPA |— sipen.css ← design system global (variáveis, componentes) |— manifest.json ← PWA manifest (ícones, nome, display) |— sw.js ← Service Worker (cache offline) | |— core/ ← infraestrutura da aplicação | |— api.js ← SUPABASE_URL, headers, fetch helpers | |— auth.js ← perfis, permissões, sessão | |— init.js ← bootstrap: auth check, nav, módulos | |— router.js ← CRUMB map, rotas hash (#modulo-view) | |— theme.js ← dark/light mode, variáveis CSS | |— ui.js ← toast T(), modal helpers | |— pwa.js ← registro do Service Worker | |— modules/ ← cada pasta = 1 módulo de negócio | |— agenda/ view.html + index.js | |— membresia/ view.html + index.js + core.js | |— financeiro/ view.html + index.js + cnab.js | |— conselho/ view.html + pautas.js + atas.js | |— demandas/ view.html + index.js | |— pastoral/ view.html + index.js + rede-cuidado.js | |— whatsapp/ config.js + index.js + tab.js | |— infraestrutura/ view.html | |— juridico/ view.html + contratos.js | |— ...18 módulos no total | |— views/ ← fragmentos HTML do shell | |— main.html ← área de conteúdo central | |— sidebar.html ← menu lateral com navegação | |— login.html ← tela de login | |— modals.html ← modais globais (wa-resp-modal, etc.) | |— supabase/ | |— functions/ ← Edge Functions (Deno/TypeScript) | | |— whatsapp-receive/index.ts | | |— whatsapp-send/index.ts | | |— whatsapp-status/index.ts | | |— email-send/index.ts | | |— gdrive-upload/index.ts | |— migrations/ ← DDL: CREATE TABLE, RLS, indexes | |— patches/ ← scripts DML: seeds, fixes pontuais | |— scripts/ ← utilitários Node.js (sync auth users)

2.2 Camada Core

Arquivo	Responsabilidade	Globals Expostos
core/api.js	URL do Supabase, anon key, funções de fetch autenticado, cliente supabase-js	SUPABASE_URL, SUPABASE_ANON_KEY, apiHeaders(), apiBaseUrl(), getSupabase()
core/auth.js	Perfis (11), matriz de permissões, login, logout, verificação de nível de acesso	PERFIS, PERMISSOES_MATRIZ, USUARIO_ATUAL, sipenLogin(), sipenLogout(), podeFazer()
core/router.js	Mapa de rotas (CRUMB), roteamento por hash URL, breadcrumbs	CRUMB, MC (cores), navegarPara(rota)
core/ui.js	Sistema de toast, modais genéricos, formatação de datas e valores	T(titulo, msg), escapeHtml(), fmtMoeda(), fmtData()
core/init.js	Bootstrap: verifica sessão, carrega sidebar, injeta módulo correto	-
core/theme.js	Dark/light mode, variáveis CSS dinâmicas, preferência persistida	toggleTheme(), THEME_ATUAL

2.3 Padrão de Módulo

Cada módulo segue uma estrutura consistente:

```
// modules/<módulo>/index.js // IIFE - escopo
isolado, sem vazamentos (function () { //
estado local do módulo let _cache = null; //
funções privadas async function _load() { ...
} function _render() { ... } // API pública
via window.* window.moduloInicia = async
function() { await _load(); _render(); }; })
();
```

Convenções obrigatórias

- IIFE para isolamento de escopo
- `_api()` e `_headers()` via wrappers locais
- `_toast()` via wrapper local
- Funções expostas via `window.*` somente quando chamadas pelo HTML
- Sem `var` — apenas `const` e `let`
- Rota registrada no CRUMB do `router.js`
- `view.html` carregada dinamicamente no shell



3. Módulos do Sistema



Dashboard

Visão executiva consolidada: KPIs globais, atividade recente, alertas e acesso rápido a todos os módulos.

dashboard

KPIs

alertas



Membresia

Cadastro completo de membros, histórico de transferências, aniversários, relatórios de membresia ativa/inativa.

pessoas

membros

transferência

relatórios



Financeiro

Lançamentos, fluxo de caixa, contas a pagar/receber, categorias, CNAB 240 (remessas bancárias), auditoria financeira.

lançamentos

CNAB 240

fluxo de caixa

auditoria



Agenda

Agendamento de espaços, cultos e eventos. Calendário visual, confirmações e lembretes via WhatsApp.

agendamentos

cultos

WhatsApp



Demandas

Sistema de solicitações internas com 15 categorias, triagem automática por área, acompanhamento de status e notificações WA.

solicitações

15 categorias

WhatsApp

protocolo



Conselho e Governança

Reuniões, pautas e deliberações. Controle de nomeados, ordenados, seminaristas, contratados e membros da comissão executiva. Notificações WA.

reuniões

pautas

nomeados

WhatsApp



Pastoral

Rede de cuidado pastoral, acompanhamento de famílias, pedidos de oração, visitas pastorais e relatórios de atendimento.

cuidado

visitas

oração



Departamentos / Ministerial

Gestão de ministérios, setores, escalas de serviço, membros por ministério, convocações e avisos via WhatsApp.

ministérios

escalas

setores



Infraestrutura e Conservação

Ordens de serviço de manutenção, controle de estoque de materiais, abertura e conclusão de chamados.

manutenção

estoque

chamados



Jurídico

Contratos, pareceres jurídicos, processos, prazos, riscos e documentos. Gestão de instrumentos legais da igreja.

contratos

pareceres

processos



Junta Diaconal

Diáconos, escalas diaconais, famílias assistidas, ação social/beneficência, visitação e patrimônio operacional.

diáconos

escalas

ação social



Congregações

Gestão de congregações vinculadas, líderes, cultos, agenda e lançamentos financeiros por congregação.

congregações

cultos

líderes



Pequenos Grupos (PGs)

Gerenciamento de PGs, líderes, participantes, encontros, estudos bíblicos, visitantes e pedidos de oração.

discipulado

encontros

estudos



Eventos

Cadastro e gestão de eventos institucionais, inscrições online, controle de vagas e lista de participantes.

eventos

inscrições

vagas



Relatórios

Geração de relatórios consolidados por módulo: financeiro, membresia, demandas e atividades ministeriais.

relatórios

exportação

consolidados



Administrativo

Secretaria, RH, estoque, controle de estacionamento, acesso facial biométrico, participantes externos e documentos.

secretaria

RH

acesso facial

estoque



Comunicação

Solicitações de criação de arte, gestão de conteúdo para redes sociais e comunicados internos.

design

comunicados

redes sociais



WhatsApp

Configuração centralizada do módulo WA: responsáveis por módulo, templates, log de envios, ativação/pausa por módulo.

configuração

responsáveis

templates

log



Configurações

Configurações do sistema: usuários, perfis, parâmetros globais e secrets de integração (via Supabase).

admin

usuários

sistema



Projetos

Portfólio de projetos institucionais, etapas, progresso, responsáveis e acompanhamento de prazo.

projetos

etapas

portfólio



Área do Membro

Portal self-service para membros: ver dados, solicitar atualizações, acompanhar demandas e notificações.

self-service

perfil

demandas



Administração de Usuários

CRUD de usuários do sistema, vinculação com pessoas cadastradas, controle de perfis e senhas.

usuários

perfis

admin



4. Perfis e Permissões

Perfis do Sistema

Perfil	Nome	Nível	Descrição
admin_geral	Administrador Geral	7	Acesso total ao sistema
conselho	Membro do Conselho (Supervisor)	6	Supervisão de governança e relatórios
pastoral	Pastoral	5	Gestão pastoral completa
adm_operacional	Adm. Operacional	4	Administrativo, financeiro, agenda
lider_ministerio	Líder de Ministério	3	Coordenação de departamentos
lider_area	Líder de Área (Líder Setorial)	3	Responsável por setor específico
membro_ministerio	Membro de Ministério (Sacerdote)	2	Participação em ministérios
lider_congregacao	Líder de Congregação	2	Gestão de congregação vinculada
operacional_servicos	Operacional Serviços (Contratados)	1	Acesso operacional restrito
membro_congregacao	Membro de Congregação	1	Acesso básico congregação
membro_igreja	Membro da Igreja	0	Área do membro apenas

Matriz de Permissões por Módulo

FULL = leitura e escrita completa | **READ** = somente leitura | **RESTR.** = restrito (ver somente próprios registros) | **—** = sem acesso

Módulo	Admin	Conselho	Pastoral	Adm.Op.	Líder Min.	Mb. Min.	Op. Serv.	Mb. Igreja
Administrativo	FULL	READ	READ	FULL	—	—	—	—
Financeiro	FULL	READ	—	FULL	—	—	—	—
Jurídico	FULL	READ	—	RESTR.	—	—	—	—
Pastoral	FULL	READ	FULL	—	—	—	—	—
Conselho / Gov.	FULL	FULL	READ	—	—	—	—	—
Departamentos	FULL	READ	READ	READ	FULL	RESTR.	—	—
Agenda	FULL	READ	FULL	FULL	FULL	RESTR.	—	—
Demandas	FULL	READ	READ	FULL	RESTR.	RESTR.	RESTR.	—

Módulo	Admin	Conselho	Pastoral	Adm.Op.	Lider Min.	Mb. Min.	Op. Serv.	Mb. Igreja
Membresia	FULL	READ	FULL	FULL	READ	—	—	—
Infraestrutura	FULL	READ	—	FULL	—	—	RESTR.	—
Área do Membro	FULL	RESTR.	RESTR.	RESTR.	RESTR.	RESTR.	RESTR.	RESTR.
Configurações	FULL	—	—	—	—	—	—	—



5. Banco de Dados (Supabase)

5.1 Tabelas Principais

Tabela	Módulo	Descrição
peessoas	Core	Registro central de todas as pessoas (membros, visitantes, externos). Chave estrangeira em quase todas as tabelas.
user_profiles	Auth	Perfil do usuário autenticado: role, ativo, pessoa_id vinculada, permissões personalizadas.
membros	Membresia	Dados de membresia: data de ingresso, tipo, status, congregação, histórico de transferências.
demandas	Demandas	Solicitações com protocolo gerado, area, status, prioridade, financeiro_data (para demandas financeiras).
demanda_andamentos	Demandas	Histórico de movimentações/comentários de cada demanda.
financeiro	Financeiro	Lançamentos de receita e despesa, categorias, status de pagamento, competência.
financeiro_solicitacoes	Financeiro	Solicitações de pagamento/reembolso com aprovação e anexos.
cnab_remessas / cnab_retornos	Financeiro	Arquivos CNAB 240 para pagamentos bancários.
agenda	Agenda	Agendamentos de espaços: título, tipo, data, horário, espaço, status de confirmação.
conselho_reunioes	Conselho	Reuniões do conselho: tipo (Ordinária/Extraordinária/Comissão), data, status.
conselho_pautas	Conselho	Itens de pauta com ordem, categoria, status, deliberação e histórico.
conselho_pautas_historico	Conselho	Auditoria de todas as alterações em pautas.
nomeados / oficiais / seminaristas / contratados	Conselho	Pessoas com funções especiais no conselho.
ministerios / ministerio_membros	Departamentos	Ministérios da igreja e seus membros com funções e status.
ministerio_setores / ministerio_setor_membros	Departamentos	Setores dentro de cada ministério.
escala_pregacao / escala_diaconal	Ministerial	Escalas de pregação e serviço diaconal.
eventos / evento_inscricoes	Eventos	Eventos institucionais e controle de inscrições com status.
pgs / pg_participantes / estudos_pgs	PGs	Pequenos grupos, participantes e encontros de estudo.

Tabela	Módulo	Descrição
congregacoes / congregacao_cultos	Congregações	Congregações vinculadas e seus cultos regulares.
contratos / contrato_anexos	Jurídico	Instrumentos contratuais com partes, valores, vigência e documentos.
projetos / projeto_etapas	Projetos	Portfólio de projetos institucionais com etapas e progresso.
estoque_itens / estoque_movimentacoes	Administrativo	Controle de materiais com entradas, saídas e saldo.
acesso_facial	Administrativo	Log de acesso biométrico (facial recognition).
rede_cuidado	Pastoral	Rede de cuidado pastoral com grupos e responsáveis.
participantes_externos	Administrativo	Pessoas sem membresia que participam de eventos/cultos.
whatsapp_config	WhatsApp	Configuração global do WA: URL BotConversa, ativo.
whatsapp_mensagens	WhatsApp	Log de todas as mensagens enviadas com status, idempotency_key e referência.
whatsapp_templates	WhatsApp	Templates de mensagem por módulo.
whatsapp_modulo_config	WhatsApp	Configuração por módulo: ativo/pausado, descrição.
whatsapp_modulo_responsaveis	WhatsApp	Pessoas responsáveis por módulo WA (única fonte de verdade).
whatsapp_ia_log	WhatsApp	Log de processamento Gemini: mensagem recebida, resposta IA, demanda criada.
email_notificacoes	E-mail	Fila e log de notificações enviadas por e-mail.
logs_sistema	Core	Auditoria geral de ações do sistema.

5.2 Row Level Security (RLS)

Todas as tabelas têm RLS habilitado. As políticas seguem o padrão:

Usuários autenticados

- SELECT: permitido para autenticados
- INSERT/UPDATE/DELETE: verificado via `public.is_admin()` ou permissão específica

Service Role

- Bypass total de RLS
- Usado pelas Edge Functions com `service_role_key`

Nota: A função `public.is_admin()` verifica se o `auth.uid()` tem `role = 'admin'` em `user_profiles`. Ela é usada em todas as políticas de escrita.



6. Edge Functions (Serverless)

As Edge Functions rodam no runtime Deno do Supabase. O deploy é feito manualmente via CLI:

```
supabase functions deploy <nome-da-function> --project-ref erhwyfzpycahgsohhbh
```

Atenção: `git push` NÃO faz deploy das Edge Functions. O deploy deve ser feito explicitamente via Supabase CLI.

Funções Disponíveis

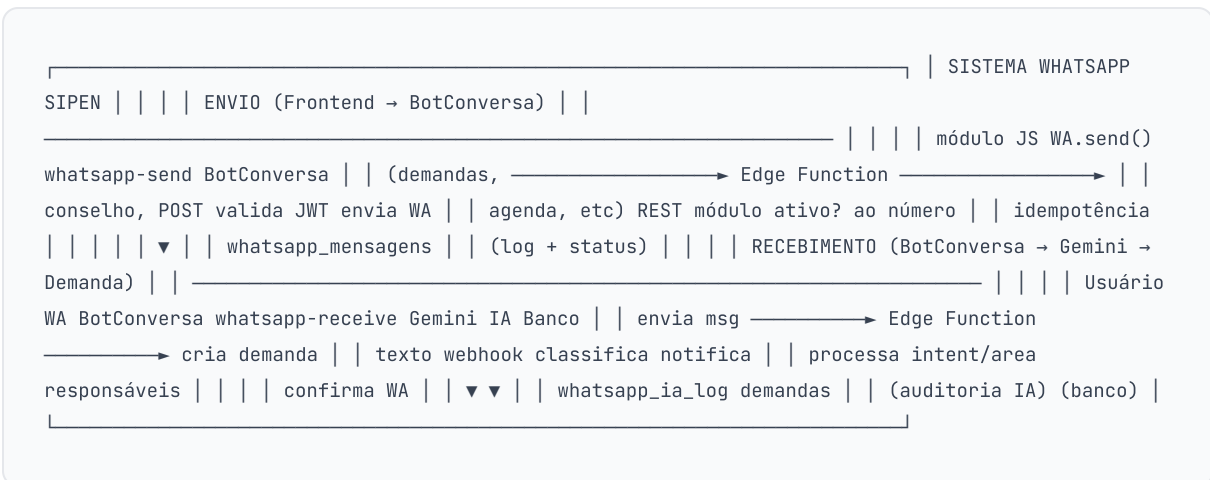
Função	Trigger	Responsabilidade	Secrets Necessários
whatsapp-send	Frontend (POST)	Valida JWT, verifica módulo ativo, implementa idempotência via whatsapp_mensagens, chama BotConversa API para envio.	BOTCONVERSA_API_KEY, BOTCONVERSA_URL
whatsapp-receive	Webhook BotConversa	Recebe mensagens WA, classifica via Gemini IA, cria demanda no banco, notifica responsáveis do módulo, confirma ao remetente.	BOTCONVERSA_API_KEY, BOTCONVERSA_URL, GEMINI_API_KEY, SUPABASE_SERVICE_ROLE_KEY
whatsapp-status	Frontend (GET)	Verifica status da conexão WhatsApp com BotConversa (conectado/desconectado).	BOTCONVERSA_API_KEY, BOTCONVERSA_URL
email-send	Frontend / Edge	Envio de e-mails transacionais via SMTP (Nodemailer). Loga em email_notificacoes.	SMTP_HOST, SMTP_USER, SMTP_PASS
gdrive-upload	Frontend (POST)	Upload de arquivos para o Google Drive da organização via Service Account.	GDRIVE_SERVICE_ACCOUNT_JSON

Estrutura de whatsapp-send

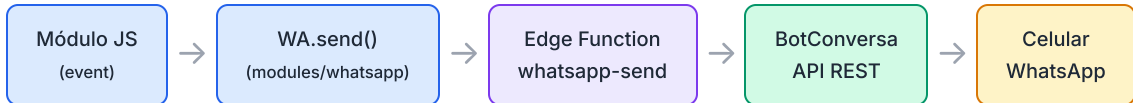
```
// POST /functions/v1/whatsapp-send // Body: { para, nome, mensagem, modulo, referenciaT, referenciaId, chave }
1. Valida JWT do usuário (Authorization: Bearer <token>)
2. Verifica whatsapp_modulo_config.ativo = true para o módulo
3. Tenta INSERT em whatsapp_mensagens com idempotency_key = chave → Se 23505 (unique violation): mensagem já enviada, retorna 200 sem reenvio
4. Chama BotConversa REST API com número + mensagem
5. Atualiza status: "enviado" ou "erro" no log
```

7. Integração WhatsApp

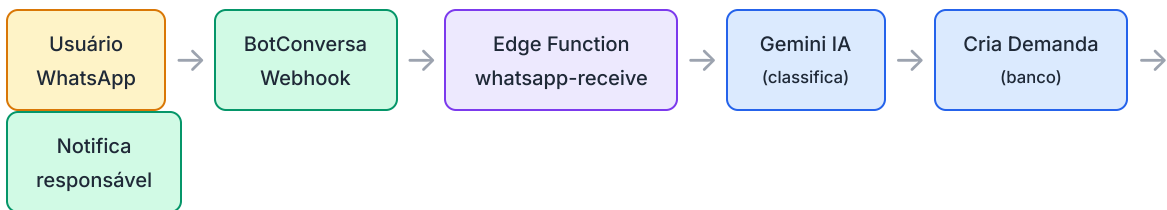
7.1 Arquitetura Geral



7.2 Fluxo de Envio



7.3 Fluxo de Recebimento (IA)



O Gemini analisa a mensagem de texto e retorna JSON estruturado com:

```
{ "intent": "criar_demanda" | "consultar" | "outro", "area_id": "financeiro" | "manutencao" | "conselho" | ..., "titulo": "Título extraído da mensagem", "descricao": "Descrição detalhada", "prioridade": "NORMAL" | "ALTA" | "URGENTE", "financial_data": { // opcional, somente financeiro "valor": 1500.00, "data_vencimento": "2026-06-01" }, "resposta": "Mensagem de confirmação ao usuário" }
```

7.4 Módulos com Notificações WA

Módulo WA	Evento que dispara	Código / Função
DEMANDAS	Nova demanda criada (qualquer área)	<code>_notificarResponsaveisWA()</code> em <code>demandas/index.js</code>
CONSELHO	Nova reunião criada	<code>_notificarConselhoWA()</code> em <code>conselho/pautas.js</code>
CONSELHO	Nova pauta adicionada	<code>_notificarConselhoWA()</code> em <code>conselho/pautas.js</code>
AGENDA	Novo agendamento / confirmação	<code>WA.send()</code> em <code>agenda/index.js</code>
FINANCEIRO	Demanda financeira (via DEMANDAS)	<code>_notificarResponsaveisWA()</code> com <code>modulo=FINANCEIRO</code>
INFRAESTRUTURA	Demanda de manutenção (via DEMANDAS)	<code>_notificarResponsaveisWA()</code> com <code>modulo=INFRAESTRUTURA</code>
PASTORAL	Pedido de oração/visitação (via DEMANDAS)	<code>_notificarResponsaveisWA()</code> com <code>modulo=PASTORAL</code>
MEMBRESIA	Boas-vindas novo cadastro	<code>WA.send()</code> em <code>membresia</code>

7.5 Tabelas WhatsApp

Tabela	Finalidade	Campos chave
<code>whatsapp_modulo_config</code>	Ativa ou pausa envio por módulo	<code>modulo</code> (PK), <code>ativo</code> , <code>descricao</code>
<code>whatsapp_modulo_responsaveis</code>	Quem recebe notificações de cada módulo (fonte única)	<code>modulo</code> , <code>pessoa_id</code> , <code>ativo</code> (UNIQUE <code>modulo+pessoa_id</code>)
<code>whatsapp_mensagens</code>	Log de todos os envios com idempotência	<code>para_numero</code> , <code>mensagem</code> , <code>modulo</code> , <code>referencia_tipo</code> , <code>referencia_id</code> , <code>status</code> , <code>idempotency_key</code> (UNIQUE)
<code>whatsapp_ia_log</code>	Auditoria de processamento Gemini	<code>numero_origem</code> , <code>mensagem_original</code> , <code>resposta_ia</code> , <code>demanda_id</code> , <code>status</code>
<code>whatsapp_config</code>	Configuração global (URL, instância)	<code>chave</code> , <code>valor</code> (configurações BotConversa)

Regra de responsáveis: A tabela `whatsapp_modulo_responsaveis` é a **única fonte de verdade** para decidir quem recebe notificações. A tabela legada `demanda_responsaveis` foi descontinuada para fins de notificação WA. Fallback: admins (`user_profiles.role = 'admin'`) quando nenhum responsável está configurado.

Módulos WA Disponíveis

AGENDA Confirmações e lembretes	DEMANDAS Status de solicitações	FINANCEIRO Pagamentos e avisos	PASTORAL Comunicações pastorais
MINISTERIAL Escalas e convocações	MEMBRESIA Boas-vindas e cadastro	CONSELHO Reuniões e pautas	JUNTA_DIACONAL Atendimentos diaconais

INFRAESTRUTURA

Ordens de manutenção

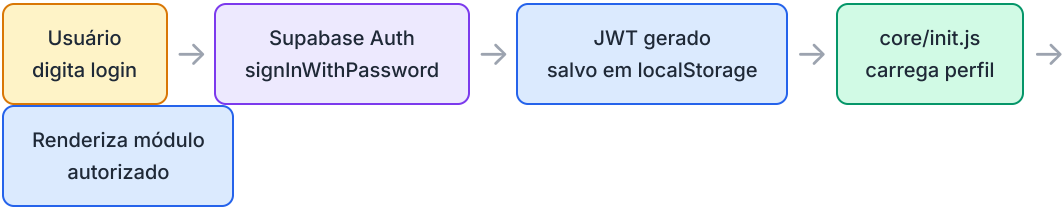
JURIDICO

Prazos e processos



8. Autenticação e Segurança

Fluxo de Autenticação



Aspecto	Implementação
Autenticação	Supabase Auth com email/senha. JWT armazenado em localStorage com key sipen_auth_session.
Autorização	Frontend verifica PERMISSOES_MATRIZ em core/auth.js antes de renderizar. Backend aplica RLS em todas as tabelas.
RLS	Todas as tabelas têm Row Level Security habilitado. Políticas usam auth.uid() e public.is_admin().
Service Role	Edge Functions usam SUPABASE_SERVICE_ROLE_KEY (secret) para operações administrativas. Nunca exposta ao frontend.
Secrets	Todas as chaves de API (BotConversa, Gemini, SMTP, Google Drive) são Supabase Secrets. Acessíveis via Deno.env.get() nas Edge Functions.
Anon Key	A chave anon é pública (bundle JS) e só permite operações explicitamente autorizadas pelas políticas RLS para usuários não autenticados.
Sessão	Sessão persistida no localStorage. Token renovado automaticamente pelo Supabase JS SDK. Logout limpa todos os dados da sessão.

Pontos de Segurança

XSS: Todas as saídas HTML passam pela função `escapeHtml()` disponível globalmente via `core/ui.js`. Módulos têm wrappers locais `_eh()` e `_ea()`.

Injeção SQL: Todas as queries usam parâmetros da Supabase REST API (query string URL-encoded ou JSON body). Sem SQL dinâmico no frontend.

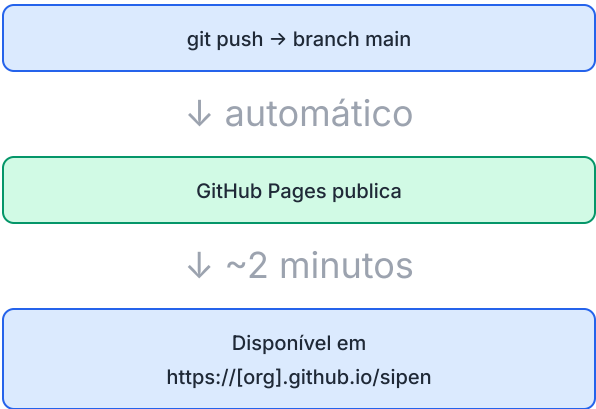
CSRF: Requisições autenticadas exigem o JWT no header `Authorization: Bearer`. A API REST do Supabase não usa cookies de sessão.



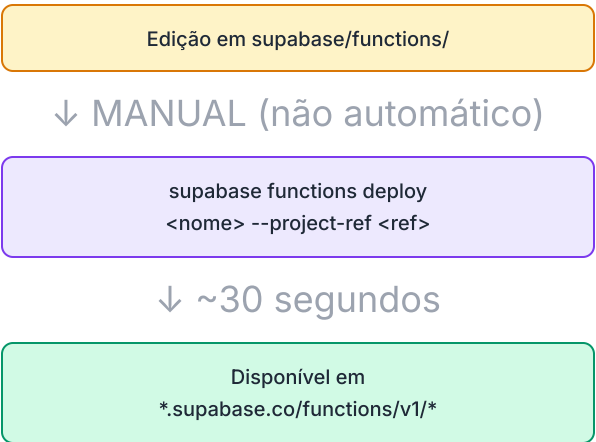
9. Deploy e Infraestrutura

Pipeline de Deploy

Frontend (GitHub Pages)



Edge Functions (Supabase)



Infraestrutura

Serviço	Plano / Detalhe	URL
GitHub Pages	Gratuito — hospedagem estática com HTTPS automático	github.io/sipen
Supabase	Projeto: <code>erhwryfzpycahgsohhbh</code> — PostgreSQL, Auth, Storage, Edge Functions	erhwryfzpycahgsohhbh.supabase.co
BotConversa	API REST para WhatsApp Business. Webhook configurado para a Edge Function <code>whatsapp-receive</code>	API BotConversa
Google Gemini	<code>gemini-2.5-flash-lite</code> — classificação de mensagens WA. Chave via Supabase Secret <code>GEMINI_API_KEY</code>	generativelanguage.googleapis.com
Google Drive	Service Account para upload de documentos. Chave via Supabase Secret <code>GDRIVE_SERVICE_ACCOUNT_JSON</code>	drive.googleapis.com
SMTP	Servidor de e-mail para notificações. Configurado via Supabase Secrets	Secrets: <code>SMTP_*</code>

PWA (Progressive Web App)

O SIPEN é instalável como app no celular (iOS e Android) via **manifest.json** + **sw.js**. O Service Worker faz cache dos assets estáticos para uso offline. Dados do banco requerem conexão.

Migrations e Patches

Tipo	Pasta	Como executar	Finalidade
Migrations	supabase/migrations/	Supabase Dashboard → SQL Editor	CREATE TABLE, índices, RLS, políticas — mudanças estruturais
Patches	supabase/patches/	Supabase Dashboard → SQL Editor	Seeds, correções pontuais de dados, ajustes de configuração



10. Referências e Configurações

Identificadores do Projeto

Recurso	Valor
Supabase Project Ref	<code>erhwryfzpycahgsohhbh</code>
Supabase API URL	<code>https://erhwryfzpycahgsohhbh.supabase.co</code>
Supabase Dashboard	<code>https://supabase.com/dashboard/project/erhwryfzpycahgsohhbh</code>
Modelo IA	<code>gemini-2.5-flash-lite</code>
Versão do Sistema	<code>v6.30.36</code>

Secrets do Supabase (Edge Functions)

Secret	Usado por	Descrição
<code>BOTCONVERSA_API_KEY</code>	whatsapp-send, whatsapp-receive, whatsapp-status	Chave de API do BotConversa para autenticação
<code>BOTCONVERSA_URL</code>	whatsapp-send, whatsapp-receive, whatsapp-status	URL base da API BotConversa (instância)
<code>GEMINI_API_KEY</code>	whatsapp-receive	Chave Google Generative AI para classificação
<code>SUPABASE_SERVICE_ROLE_KEY</code>	whatsapp-receive	Chave service_role para bypass de RLS
<code>SMTP_HOST</code> / <code>SMTP_USER</code> / <code>SMTP_PASS</code>	email-send	Credenciais do servidor de e-mail
<code>GDRIVE_SERVICE_ACCOUNT_JSON</code>	gdrive-upload	Service Account Google para upload de arquivos

Padrão de Versionamento

O sistema segue **Semantic Versioning** informal: `v{major}.{minor}.{patch}`. Commits seguem Conventional Commits: `feat:`, `fix:`, `refactor:`, `chore:`. Cada alteração gera um commit dedicado.

Manutenção e Pontos de Atenção

AREA_MODULO mapping: Existe em dois lugares — `supabase/functions/whatsapp-receive/index.ts` (usa `area_id`) e `modules/demandas/index.js` (usa `area_nome`). Ambos devem ser mantidos em sincronia quando novas áreas/módulos forem adicionados.

Edge Functions: Alterações no código TypeScript não entram em produção com `git push`. Sempre executar `supabase functions deploy` após edições.

Responsáveis WA: A tabela `whatsapp_modulo_responsaveis` deve ter ao menos um registro ativo por módulo. Se vazia, o sistema usa `admins` como fallback, o que pode gerar notificações em excesso para a liderança.

SIPEN — Sistema Integrado de Gestão · IPPenha

Documentação gerada em maio de 2026 · Versão v6.30.36

CONFIDENCIAL · Uso Interno